

# Matt Kruger · HW6

1. Give an implementation-level description of a Turing Machine that decides the language  $\{c^{n^2} \mid n \geq 0\}$ . Note that the input alphabet is  $\{c\}$ . (10 points)

By implementation-level description, we don't mean the state diagram, but a higher level that still refers to key details of the tape content and head movement. For illustration, see the descriptions given in Examples 3.11 and 3.12 in Section 3.1. Feel free to use a multi-tape TM if needed.

$\{c^{n^2} \mid n \geq 0\} \Rightarrow$  the count of c's is a perfect square

The TM  $M$  operates on the input string  $w$  as follows:

1. Scan the input tape left to right. If the first symbol is a blank  $\sqcup$  the input is  $c^0 = \epsilon \Rightarrow$  **ACCEPT**. If the first symbol is a  $c$ , return the head to the left end of the tape, move forward
2. Mark the first  $c$  on the tape with  $X$  to signify the subtraction of the first odd number, 1. Set the current odd number to be subtracted by 3
3. Start from the  $X$  marker and scan to the right, cross off the number of  $c$ 's equal to the current odd number using  $Y$ . The TM must keep track of the number of  $c$ 's it has crossed

Case a) Successful subtraction:

- If the TM successfully crosses off the required number of  $Y$ 's and next symbol is  $\sqcup \Rightarrow$  **ACCEPT**

Case b) too few  $c$ 's:

- if the TM runs into a blank  $\sqcup$  before crossing off the required number of  $c$ 's, then the number of  $c$ 's not perfect square  $\Rightarrow$  **REJECT**

Case c) more  $c$ 's remain

- if the TM successfully crosses off required number of  $c$ 's and there are still  $c$ 's:
  1. inc current odd number by 2
  2. return tape head to leftmost  $X$ , proceed to 4

4. Starting from the  $X$  marker, scan to the right and change all  $Y$ 's back to  $c$ 's. Mark the leftmost uncrossed  $c$  with  $X$ . Return the head to this new marker. Proceed to step 3 with new incremented odd number

2. A Turing-machine with left reset is similar to an ordinary Turing Machine, but the transition function has the form

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{R, \text{RESET}\}$$

If  $\delta(q, a) = (r, b, \text{RESET})$ , when the machine is in state  $q$  reading an  $a$ , the machine's head jumps to the left-hand end of the tape after it writes  $b$  on the tape, and the machine enters state  $r$ . Note that such a machine does not have the ability to move the head one symbol left.

- (a) Show that if a language is recognized by a TM with left reset, then it is also recognized by an ordinary TM. (5 points)
- (b) Show that if a language is recognized by an ordinary TM, then it is also recognized by a TM with left reset. (10 points)

$M_{LR} = \text{left reset TM}$

a) An ordinary TM can simulate  $M_{LR}$ :

• if  $M_{LR}$  executes  $\delta(q, a) = (r, b, R)$ ,  $M$  writes  $b$  and then moves the head one cell right. Then enter state  $r$ .

simulate

• if  $M_{LR}$  executes  $\delta(q, a) = (r, b, \text{RESET})$ ,  $M$  writes  $b$ , then repeatedly moves the head one cell left until it reaches the left end marker of the tape. Then enter state  $r$ .

Because an ordinary TM can simulate  $M_{LR}$  by nearly one step it implements RESET by repeatedly jumping. Thus, any computation by  $M_{LR}$  is also done by an ordinary TM. Thus any language recognized. i.e. Ordinary

b) Similarly, a TM with left reset can simulate an ordinary TM by marking the tape cells that represents the head position. Right moves will be identical from TM and  $T_{MR}$ , however, left moves are repeated to simulate RESET.

Because every left move and right move of an ordinary TM can be simulated this way, the left reset TM recognizes the same language

3. Let  $ALL_{DFA} = \{\langle A \rangle \mid A \text{ is a DFA and } L(A) = \Sigma^*\}$ . Here,  $L(A)$  refers to the language recognized by DFA  $A$ , and  $\Sigma$  refers to the input alphabet of  $A$ . Show that  $ALL_{DFA}$  is decidable. (12 points)

$$L(A) = \Sigma^* \text{ iff } \overline{L(A)} = \emptyset$$

if DFA accepts some string iff an accept state is reachable  $\Rightarrow$  Decidable

Design M for  $ALL_{DFA}$ :

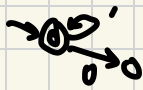
1. on input  $\langle A \rangle$ ,  $A = (Q, \Sigma, \delta, q_0, F)$ :
2. Construct the complement of  $A$ ,  $A'$  by inverting accept/reject states
3. Perform BFS on transition graph of  $A'$  starting at  $q_0$
4. if an accept state is reached in  $A' \Rightarrow$  **reject** because  $\overline{L(A)} \neq \emptyset$
5. if no accepting state is reached  $\Rightarrow$  **accept** because  $\overline{L(A)} = \emptyset$ , thus  $L(A) = \Sigma^*$

using this procedure,  $ALL_{DFA}$  is decidable

4. Let  $L_1 = \{ \langle R \rangle \mid R \text{ is a regular expression over alphabet } \{0,1\}, \text{ and } R \text{ describes a language that contains at least one string with no } 0 \}.$  Show that  $L_1$  is decidable. (13 points)

$$\Sigma = \{0,1\} \quad 1^* = \{ \epsilon, 1, 11, 111, \dots \}$$

Decide  $n$  on  $L_1$ :

1. Convert the regular expression  $R$  to an NFA  $N_R$
2. Build a DFA  $D_{1^*}$  for  $1^*$  that has self loop as start and accept state  

3. Construct DFA  $A$  that recognizes  $L(N_R) \cap L(D_{1^*})$  intersection of  $N_R$  on a NFA and DFA. This produces an automaton that is finite
4. Perform BFS on  $A$  to test non-emptiness:
  - if an accept state is reachable  $\Rightarrow$  **accept**
  - if no accept state  $\Rightarrow$  **reject**

$L(R) \cap 1^*$  i.e. contains string without 0

Thus  $A$  accepts exactly the strings BOTH in  $L(R)$  and  $1^*$  to decide non-emptiness. Because every construction is effective and finite this procedure decides  $L_1$ .